

SE-MPPI: Coordinating Online Local-Minima Escape with Control-Barrier-Function Safety in a Nav2-Native Controller

Jungmo Kang¹

¹Independent Researcher, Seoul, Republic of Korea

Model Predictive Path Integral (MPPI) control, as shipped in the ROS 2 Nav2 stack, is a widely used deployable local controller for mobile robots, yet in narrow, cluttered and dynamic spaces it can stall in local minima, and its critics avoid collisions only through soft costs, without a formal safety guarantee. Methods exist for each weakness, and several works outside Nav2 address both together, but we know of no public Nav2 controller plugin that combines local-minima escape with a barrier-based safety filter. This system paper presents *SE-MPPI* (Safe-Escape MPPI), a Nav2 controller plugin that integrates online local-minima detection-and-escape for MPPI with a control-barrier-function (CBF) safety filter for tracked dynamic obstacles. One path-progress entrapment signal switches repulsion and free-space gap costs into the MPPI rollouts and, through a coordinator, raises the barrier’s class- $\mathcal{K}$ gain. By a standard argument for time-varying CBF gains, this schedule preserves forward invariance of the barrier’s safe set, a conditional guarantee that covers only a look-ahead point and the tracked dynamic obstacles, assumes that the barrier inequality holds between control updates, and requires the quadratic program to use no slack, a condition that the benchmark QP violates at least once in most trials with moving obstacles. In a randomized, paired 1,200-trial benchmark run on a Python 2D mirror of the controller’s primitives, not on the Nav2 controller itself, the detection-and-escape layer lifts success from 0% to 88–90% on a static U-trap family and from 0% to 62–78% on a narrow-dynamic family (Holm-adjusted McNemar $p = 1.4 \times 10^{-9}$ and 1.1×10^{-6}), and disabling its free-space gap search returns both families to 0%. The escape–safety coordination, by contrast, is statistically indistinguishable from an independent escape+CBF stack; we report this null and claim the coordination only as a mechanism, not as an empirical gain. The controller also loads and runs in a live ROS 2 Jazzy, Nav2 and Gazebo stack, where it exposed three deployment faults that unit tests could not, although no live run has yet reached its goal. Code, raw trials and regeneration scripts: <https://github.com/kjungmo/se-mppi-nav2>.

Keywords: model predictive path integral control, control barrier functions, local minima, collision avoidance, ROS 2 Nav2

1. Introduction

Autonomous mobile robots increasingly work in cluttered, dynamic indoor spaces such as warehouses, hospitals and service floors, where the local controller must be fast, non-conservative and safe at the same time. Within the ROS 2 Nav2 ecosystem (Macenski et al., 2020), the MPPI controller (`nav2_mppi_controller`) is a widely used high-performance option: it samples a batch of control rollouts, scores them with a bank of critic plugins, and fuses them with an information-theoretic softmax (Williams et al., 2018). With the defaults we build on, it plans over a horizon of $56 \times 0.05 \text{ s} \approx 2.8 \text{ s}$ with 1,000 rollouts per cycle.

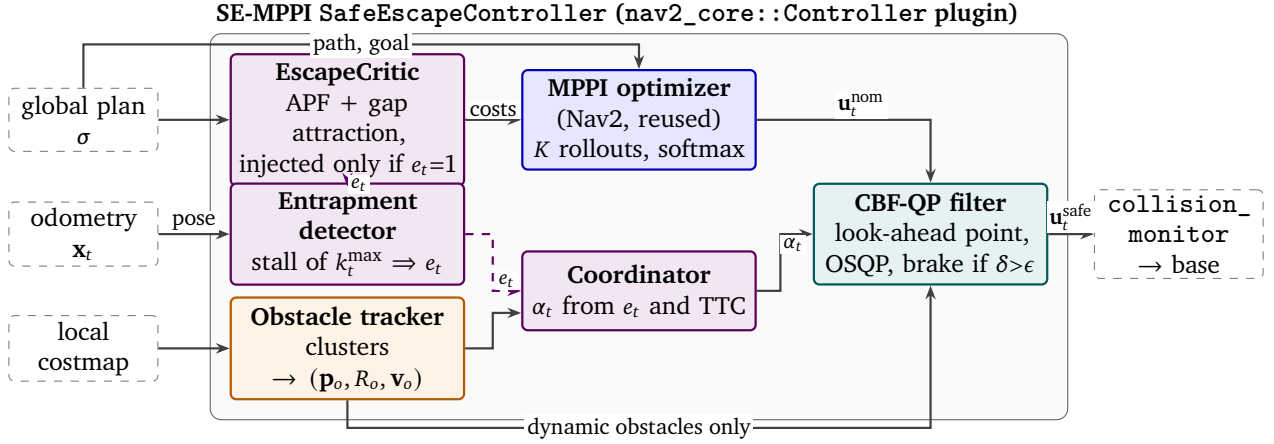


Figure 1: Overview of SE-MPPI. The stock Nav2 MPPI optimizer produces a nominal command from sampled rollouts. A single entrapment signal e_t , computed from monotone progress along the global path (Section 3.2), switches the EscapeCritic’s repulsion and gap-attraction costs into the rollout scoring (Section 3.3) and tells the coordinator to raise the CBF gain α_t unless a tracked obstacle’s time-to-collision is imminent (Section 3.6). The CBF-QP filter then projects the nominal command onto the barrier-safe set for the tracked dynamic obstacles (Section 3.5); Nav2’s collision_monitor remains the last reactive layer.

Two weaknesses recur in exactly the environments that matter most. First, *local-minima entrapment*: a finite horizon and zero-mean Gaussian sampling make the robot stall or oscillate in front of non-convex obstacles. Within the MPPI critic set, Nav2’s mitigations (PreferForwardCritic, TwirlingCritic) are always-on cost shaping that neither detects entrapment nor steers an escape. At the stack level, Nav2 does detect a stall: the controller server’s progress checker reports a failure when the robot has not moved a set distance within a set time, and the default behaviour tree then clears the costmaps and runs generic recoveries such as spinning, waiting and backing up (Macenski et al., 2020; Nav2 contributors, 2026b,c). These recoveries are not directed toward an opening. In the MPPI literature, escape methods either reshape the sampling distribution all the time (Honda et al., 2024; Mohamed et al., 2022; Sacks and Boots, 2022; Trevisan and Alonso-Mora, 2024; Wang et al., 2021) or add guidance: GP-MPPI recommends a free-space subgoal from a learned navigability surface (Mohamed et al., 2023), and repulsive-potential augmentation adds an artificial potential field to the MPPI cost (Fuke et al., 2025b), which DRPA-MPPI switches on only when it detects entrapment (Fuke et al., 2025a). None of these carries a safety certificate.

Second, *no formal safety*: in Nav2 MPPI, collision avoidance is encoded as soft costs, the costmap is a present-time snapshot with no prediction of obstacle motion, and collision_monitor is purely reactive. CBF-MPPI fusions supply safety (Borquez et al., 2025; Parwana et al., 2024, 2025; Rabiee and Hoagg, 2025; Yin et al., 2023), but none of them detects entrapment or adds an escape mechanism (GS-MPPI counters the myopic behaviour often associated with CBFs through long-term performance terms), and none ships as a Nav2 controller. Among CBF integrations with Nav2 that we could verify, the Nav2 documentation describes inserting a third-party product, the 3Laws Supervisor, which its vendor describes as based on CBFs, into the command chain after the controller (ACCESS Newswire, 2025; Nav2 contributors, 2026a); it is a separate node rather than a controller plugin, and neither source describes a mechanism for escaping local minima.

The two weaknesses also interact: conservatism for safety invites entrapment, and aggressive escape invites collision. Several works outside Nav2 and MPPI address both at once, among them model predictive control that escapes local minima while keeping a safe backup trajectory (Soloperto et al., 2023), gap-based local planners with provable collision-free properties (Asselmeier et al., 2025; Feng

et al., 2023; Xu et al., 2021), and modulated CBF filters (Xue and Figueroa, 2025) that reduce or eliminate, among moving obstacles, the spurious equilibria that CBF quadratic programs (QPs) can introduce (Reis et al., 2021), with concurrent extensions (Chen et al., 2026; Xue et al., 2026).

We present *SE-MPPI*, a Nav2 controller plugin that subclasses the stock MPPI controller and post-processes its command each cycle (Figure 1). A single entrapment detector, driven by monotone progress along the global path and running inside the controller rather than at the stack level, is the one source of truth for the whole controller. When it fires, an *EscapeCritic* switches repulsion and free-space gap-attraction costs into the rollout scoring, and a coordinator raises the gain of a look-ahead-point CBF filter so that the escape manoeuvre is admitted, unless a tracked obstacle’s time-to-collision is imminent. This is a system contribution, and we state it as such:

- *A Nav2 Controller Integrating Escape and CBF Safety*: a public Nav2 local-controller plugin that combines online local-minima detection-and-escape with a CBF safety filter for tracked dynamic obstacles, packaged as a `nav2_core::Controller` plus a pluginlib MPPI critic over a plugin-free core, and released under Apache-2.0. Each ingredient has precedent: gated repulsion in DRPA-MPPI (Fuke et al., 2025a), free-space subgoal and gap guidance in GP-MPPI (Mohamed et al., 2023) and gap-based planners (Feng et al., 2023; Xu et al., 2021), CBF filtering of MPPI commands (Parwana et al., 2024), and a supervisor for Nav2 that its vendor describes as CBF-based (ACCESS Newswire, 2025; Nav2 contributors, 2026a). What is new is their integration inside one Nav2 controller; to our knowledge no public controller plugin already provides it.
- *An Entrapment-Gated Gain Schedule*: a rule that raises the CBF class- $\mathcal{K}$ gain on detected entrapment and drops it back on an imminent time-to-collision. Online adaptation of CBF gains is established, including selecting class- $\mathcal{K}$ parameters to reduce predicted deadlock (Kim et al., 2025) and letting MPPI optimize a barrier-rate parameter (Parwana et al., 2025); our rule is a simpler, rule-based two-level instance of online gain adaptation whose specific elements are its trigger and its time-to-collision override. Its invariance property (Proposition 1) is a known result for time-varying class- $\mathcal{K}$ terms (Parwana and Panagou, 2025; Xiao et al., 2020), which we restate as a conditional lemma: it covers the look-ahead point and the tracked dynamic obstacles, assumes the barrier inequality holds between samples, and requires a slack-free QP. In our benchmark the schedule is empirically indistinguishable from an independent escape+CBF stack.
- *Paired Benchmark and Deployment Evidence*: a randomized, seed-deterministic 1,200-trial benchmark on a Python 2D mirror of the controller’s primitives (200 seeded worlds, each run under six configurations) with paired McNemar tests and Holm correction, every number regenerated from committed per-trial data, together with a live ROS 2 Jazzy, Nav2 and Gazebo integration that exposed three faults unreachable from unit tests.

We are deliberately conservative about novelty and about what we claim empirically. Fusing CBFs with MPPI is not new, and neither is gated escape, online adaptation of the CBF gain, or treating escape and safety together; the paper is best read as a system report on putting these pieces into one Nav2 controller. The measured win, in the 2D mirror, is the online detection-and-escape layer: success rises from 0% to 88–90% on the static U-trap family and from 0% to 62–78% on the narrow-dynamic family, and the gap search is load-bearing, since without it both families stay at 0% (Sections 4.2 and 4.3). In the mirror the gap acts as a temporary subgoal, close to the subgoal guidance of GP-MPPI (Mohamed et al., 2023); the gap cost of the C++ critic has not been ablated. The coordination has the conditional invariance property of Proposition 1 yet produces no measurable outcome difference in the regimes we could test, and we delimit where it should matter (Section 5). The remaining claims are Nav2-native deployment and the faults a live stack exposes; no live run has yet reached its

goal, and no stack-level or third-party baseline has been run (Section 5).

2. Related Work

MPPI and Deployable Local Controllers. Nav2 MPPI (Macenski et al., 2020; Williams et al., 2018), TEB (Rösmann et al., 2012), DWB, a Nav2 planner of the dynamic-window family (Fox et al., 1997), and Regulated Pure Pursuit (Macenski et al., 2023) are the standard deployable local controllers. MPPI’s critic interface (`mppi::critics::CriticFunction`) exposes the per-rollout trajectories, the reference path, the goal and the furthest-reached path index, which are exactly the signals needed for entrapment detection, so a custom critic can be inserted without forking the optimizer. SE-MPPI uses this interface for escape and adds an output-side safety filter, instead of replacing the optimizer. Around the controller, Nav2 already handles stalls at the stack level: the controller server runs a progress checker, by default `SimpleProgressChecker`, which reports a failure when the robot has not moved a set radius within a set time allowance, and the default behaviour tree answers with costmap clearing, spinning, waiting and backing up (Nav2 contributors, 2026b,c). SE-MPPI’s detector differs in that it runs inside the controller on progress along the path and steers the escape instead of triggering generic recoveries; the two mechanisms can coexist.

Local-Minima Escape. Most escape methods for MPPI are always-on changes of the sampling distribution: log-MPPI (Mohamed et al., 2022), Tsallis-divergence MPC (Wang et al., 2021), Stein-variational guidance (Honda et al., 2024), ancillary-controller biasing (Trevisan and Alonso-Mora, 2024), and learned sampling distributions (Sacks and Boots, 2022). Others add guidance. GP-MPPI learns a sparse-Gaussian-process model of the navigable space around the robot and recommends a free-space subgoal to the local MPPI planner, which lets the robot escape local minima without a global map (Mohamed et al., 2023). Fuke et al. add a repulsive artificial potential to MPPI to avoid local minima without global path guidance (Fuke et al., 2025b), and their DRPA-MPPI detects potential entrapment on the predicted trajectories and only then switches to a cost with repulsion away from the local minimum (Fuke et al., 2025a). DRPA-MPPI is the closest detect-and-switch method and GP-MPPI the closest subgoal method; neither has a CBF or other formal safety layer, and neither is Nav2-integrated. SE-MPPI combines both ideas in a simpler form, a path-progress stall detector gating a classical repulsive potential (Khatib, 1986) and a ray-cast gap bearing, and its safety filter runs alongside the escape rather than gating it. Gap-based local planners are closer still to the overall architecture. Potential Gap combines gap-based local navigation with artificial potential fields and restores provable collision-free properties for realistic robot models (Xu et al., 2021); Safer Gap adds nonlinear model predictive control with keyhole zeroing-barrier-function constraints and a point-wise optimization-based safety filter as the final layer (Feng et al., 2023); and Dynamic Gap extends provably collision-free gap-based navigation to dynamic environments by modelling how gaps evolve over time (Asselmeier et al., 2025). Relative to this line, SE-MPPI uses a gap only as a gated escape heuristic inside MPPI, and its barrier covers only tracked dynamic obstacles.

Escape and Safety Together. Several works treat entrapment and safety jointly. Soloperto et al. optimize an exploration trajectory together with a safe backup trajectory in a model predictive controller and prove convergence, feasibility and constraint satisfaction under partially unknown constraints (Soloperto et al., 2023). The modulated CBF-QP of Xue and Figueroa reduces or eliminates, in dynamic environments, the spurious equilibria that CBF-QP filters create (Reis et al., 2021), and was tested on Ridgeback and Fetch robots (Xue and Figueroa, 2025). Concurrent work adds learned predictive barriers for moving concave obstacles to this filter (Xue et al., 2026), and pairs a mixed-integer MPC planner with a Minkowski-difference CBF filter to mitigate the local-minimum behaviour of reactive CBF navigation in U-shaped and maze-like environments (Chen et al., 2026). These

methods mitigate or avoid minima inside the planner or the barrier itself, whereas SE-MPPI leaves the barrier unchanged and escapes through the MPPI cost. None of them is packaged as a Nav2 controller.

CBF-MPPI Safety. Three fusion styles exist as research code: CBF conditions in the rollout cost or the sampling (Shield-MPPI (Yin et al., 2023), which adds a local repair step, and BR-MPPI (Parwana et al., 2025), which imposes the CBF condition as an equality with a parametric linear class- $\mathcal{K}$ function, augments the state with its parameter and lets MPPI optimize the parameter’s rate as an extra input), an output CBF-QP filter on the MPPI command (SCBF-MPPI (Parwana et al., 2024), built on the CBFKit toolbox (Black et al., 2024)), and per-rollout safety filtering that makes every sample provably safe (GS-MPPI (Rabiee and Hoagg, 2025), DualGuard (Borquez et al., 2025), which also filters the final command). For dynamic obstacles, DPCBF (Park et al., 2025) uses a parabolic safe set that is less conservative than collision-cone barriers but is QP-only and not embedded in MPPI. Our filter builds on the classical CBF-QP (Ames et al., 2017, 2019). None of these works ships as a Nav2 controller (CBFKit builds a standalone node), and none detects entrapment or adds an escape mechanism, although GS-MPPI counters the myopic behaviour often associated with CBFs through long-term performance terms (Rabiee and Hoagg, 2025). Among Nav2 integrations of a CBF that we could verify, the official documentation describes inserting the third-party 3Laws Supervisor, which its vendor describes as based on CBFs, between the velocity smoother and the collision monitor (ACCESS Newswire, 2025; Nav2 contributors, 2026a); it acts on the controller’s output as a separate node, and neither source describes a local-minimum escape mechanism. SE-MPPI instead places its filter inside the controller and couples it to the escape layer through its gain.

Online Adaptation of the CBF Gain. Time-varying and adapted class- $\mathcal{K}$ terms are well studied. Adaptive CBFs multiply the class- $\mathcal{K}$ functions by time-varying penalty functions and prove forward invariance of the safe set (Xiao et al., 2020); optimal-decay CBF-QPs optimize the decay rate pointwise in time to keep the QP feasible (Zeng et al., 2021); and rate-tunable CBFs adapt the class- $\mathcal{K}$ functions online and derive conditions under which multiple constraints can share the control input (Parwana and Panagou, 2025). Closest to our trigger, Kim, Kee and Panagou adapt the parameters of input-constrained CBFs online with a learned ensemble model and select, among the candidates that pass their uncertainty and risk checks, the class- $\mathcal{K}$ parameters with the minimum predicted deadlock time (Kim et al., 2025); BR-MPPI lets MPPI optimize the class- $\mathcal{K}$ parameter directly (Parwana et al., 2025). Concurrent work switches nominal objectives when CBF and control-Lyapunov constraints conflict, to reduce slowdowns and deadlocks (Walia and Leahy, 2026), and schedules the gain of a frontier barrier on an occupancy grid by how well the map is explored (Paudel et al., 2026). SE-MPPI’s coordinator is a rule-based, two-level instance of this idea, triggered by the entrapment flag and overridden by time-to-collision, and its invariance property (Proposition 1) follows the same argument as the relative-degree-one case of the adaptive-CBF result (Xiao et al., 2020, Thm. 3), there stated for Lipschitz controllers.

Conformal Prediction for Safe Control. The released controller also carries an online conformal margin that inflates the barrier radius under prediction uncertainty, drawing on conformal prediction for safe planning and control (Dixit et al., 2023; Gonzales et al., 2025; Lindemann et al., 2023; Zhou et al., 2024) and on uncertainty-aware predictive barriers (Busellato et al., 2026). This margin is follow-up work and is *not* a contribution of this paper; every result reported here is isolated from it (Section 5).

Positioning. Table 1 places the reviewed systems against four factors: shipping as a Nav2 controller plugin, online local-minimum escape or avoidance, an explicit run-time safety constraint, filter or backup, and online adaptation of the class- $\mathcal{K}$ gain. No reviewed system holds all four, but several hold two, fully or in part: stock Nav2 pairs its controller plugins with stack-level stall recovery, gap-based planners, backup-trajectory MPC and modulated CBFs combine local-minimum handling with a safety

Table 1: Positioning. (a) ships as a Nav2 controller plugin; (b) online local-minimum escape or avoidance; (c) an explicit safety constraint, filter or backup enforced at run time (for DualGuard, Hamilton–Jacobi reachability; for Soloperto et al., a safe backup trajectory; for Safer Gap, zeroing barrier functions); (d) online adaptation of the CBF class- $\mathcal{K}$ gain. $\checkmark$ = holds; $\circ$ = holds in part: stock Nav2 detects stalls at the stack level and runs generic recoveries rather than a steered escape; the 3Laws Supervisor is integrated with Nav2 as a separate node after the controller, not as a controller plugin; the gap-based planners guide the robot through free-space gaps rather than escaping a detected minimum, and the provable collision-free properties of Potential Gap and Dynamic Gap follow from the planners’ gap construction rather than from an explicit run-time constraint or filter; Kim et al. reduce predicted deadlock through class- $\mathcal{K}$ parameter selection rather than escaping a detected minimum. For the 3Laws Supervisor, (b) is marked absent because neither cited source describes a local-minimum escape mechanism. [†]Concurrent work (posted in 2026). For SE-MPPI, (c) is marked in part because its barrier covers only tracked dynamic obstacles, at a look-ahead point, and its invariance argument is conditional (Proposition 1); static structure is handled only by MPPI costs. Its (b) is demonstrated only in the Python 2D mirror, not in a live Nav2 run. No reviewed system holds all four; several hold two. The review behind this table (June 2026, extended in September 2026) covered the Nav2 MPPI critic ecosystem, the Nav2 documentation, the public issue trackers of these projects, and the works cited here; it is not exhaustive.

System	Reference	(a)	(b)	(c)	(d)
Nav2 MPPI with progress checker and recoveries	Macenski et al. (2020); Nav2 contributors (2026b,c)	$\checkmark$	$\circ$	–	–
3Laws Supervisor in Nav2	ACCESS Newswire (2025); Nav2 contributors (2026a)	$\circ$	–	$\checkmark$	–
DRPA-MPPI, GP-MPPI	Fuke et al. (2025a); Mohamed et al. (2023)	–	$\checkmark$	–	–
Dynamic Gap, Potential Gap	Asselmeier et al. (2025); Xu et al. (2021)	–	$\circ$	$\circ$	–
Safer Gap	Feng et al. (2023)	–	$\circ$	$\checkmark$	–
Backup-trajectory MPC	Soloperto et al. (2023)	–	$\checkmark$	$\checkmark$	–
MILP-MPC with CBF [†] ; MCBF-QP; predictive MCBF [†]	Chen et al. (2026); Xue and Figueroa (2025); Xue et al. (2026)	–	$\checkmark$	$\checkmark$	–
DualGuard, SCBF-MPPI, GS-MPPI, Shield-MPPI	Borquez et al. (2025); Parwana et al. (2024); Rabiee and Hoagg (2025); Yin et al. (2023)	–	–	$\checkmark$	–
CBFKit, DPCBF	Black et al. (2024); Park et al. (2025)	–	–	$\checkmark$	–
BR-MPPI	Parwana et al. (2025)	–	–	$\checkmark$	$\checkmark$
Rate-tunable, adaptive, optimal-decay CBFs	Parwana and Panagou (2025); Xiao et al. (2020); Zeng et al. (2021)	–	–	$\checkmark$	$\checkmark$
Deadlock-aware parameter adaptation	Kim et al. (2025)	–	$\circ$	$\checkmark$	$\checkmark$
SE-MPPI (ours)	this paper	$\checkmark$	$\checkmark$	$\circ$	$\checkmark$

layer, and adaptive-gain CBFs and BR-MPPI pair a barrier with gain adaptation, with deadlock-aware adaptation also holding the second factor in part. The delta of SE-MPPI is therefore the conjunction inside one Nav2 controller plugin, not any single element. Its own marks are not all full: its barrier covers only tracked dynamic obstacles, and its escape is demonstrated only in the Python 2D mirror. Its gain rule differs from prior gain adaptation mainly in its trigger, and, as Section 4.3 shows, we could not measure an outcome benefit from it.

3. SE-MPPI: Coordinated Escape and Barrier Safety in One Nav2 Controller

Problem Setup. Consider a differential-drive robot with pose $\mathbf{x}_t = (x_t, y_t, \theta_t)$, position $\mathbf{p}_t = (x_t, y_t)$ and control $\mathbf{u}_t = (v_t, \omega_t) \in \mathcal{U}$, where $\mathcal{U}$ is the box $v \in [v_{\min}, v_{\max}]$, $|\omega| \leq \omega_{\max}$, following the unicycle model $\dot{x} = v \cos \theta$, $\dot{y} = v \sin \theta$, $\dot{\theta} = \omega$. A Nav2 global planner supplies a path $\sigma = (\sigma_1, \dots, \sigma_M)$. Around the robot are static structure, seen only through the local costmap, and a set $\mathcal{O}_t$ of dynamic obstacles

o with position $\mathbf{p}_o$, radius R_o and estimated velocity $\mathbf{v}_o$. Let r be the robot's inscribed radius and m a safety margin. We seek a controller that (G1) keeps making progress along σ instead of stalling in a local minimum, (G2) keeps the robot clear of the obstacles in O_t through a barrier whose forward invariance holds under stated conditions, and (G3) runs as a Nav2 controller in real time on a CPU. Formally, each control cycle approximately solves

$$\begin{aligned} \mathbf{u}_{t:t+T-1}^* = \arg \min_{\mathbf{u}_{t:t+T-1} \in \mathcal{U}^T} \mathbb{E} \left[\sum_{\tau=t}^{t+T-1} c(\mathbf{x}_\tau, \mathbf{u}_\tau; \sigma) + e_t c_{\text{esc}}(\mathbf{x}_{t:t+T}) \right] \\ \text{s.t. } h_o \text{ satisfies the CBF condition (9) at } (\mathbf{x}_t, \mathbf{u}_t) \quad \forall o \in O_t, \end{aligned} \quad (1)$$

where c is the stock Nav2 critic cost, $e_t \in \{0, 1\}$ is the entrapment flag, c_{esc} are the escape costs, and h_o is a control barrier function for obstacle o . The tension is that G1 may demand manoeuvres that approach obstacles to round a trap, whereas a fixed safety filter for G2 forbids exactly those manoeuvres. SE-MPPI splits (1): MPPI minimizes the expected cost, with escape costs switched in by e_t (Sections 3.1 to 3.3); a per-cycle QP enforces the barrier on the first control (Sections 3.4 and 3.5); and a coordinator couples the two through the barrier gain (Section 3.6). Algorithm 1 summarizes one cycle.

3.1. Sampling Nominal Commands: Reusing the Nav2 MPPI Optimizer

SE-MPPI subclasses the stock `MPPIController` and calls its optimizer unchanged. Given the previous control sequence $\bar{\mathbf{u}}_{0:T-1}$, the optimizer draws K perturbations $\epsilon_j^k \sim \mathcal{N}(\mathbf{0}, \Sigma)$, rolls out $\mathbf{u}_j^k = \text{clip}_{\mathcal{U}}(\bar{\mathbf{u}}_j + \epsilon_j^k)$ through the motion model, and scores each rollout $\mathbf{x}_{0:T}^k$ with the listed critics C_i , among which the `EscapeCritic` (Section 3.3):

$$\begin{aligned} s^k &= \sum_i C_i(\mathbf{x}_{0:T}^k) + e_t (C_{\text{apf}}(\mathbf{x}_{0:T}^k) + C_{\text{gap}}(\mathbf{x}_{0:T}^k)), \\ w^k &= \frac{\exp(-(s^k - \min_{k'} s^{k'})/\lambda)}{\sum_{k''} \exp(-(s^{k''} - \min_{k'} s^{k'})/\lambda)}, \quad \bar{\mathbf{u}}_j \leftarrow \sum_k w^k \mathbf{u}_j^k. \end{aligned} \quad (2)$$

The first element is the nominal command $\mathbf{u}_t^{\text{nom}} = (\bar{v}_0, \bar{\omega}_0)$ (Williams et al., 2018). (Nav2 additionally regularizes the control effort through a small weight γ ; we omit it for brevity.) With the released defaults, $T = 56$, $\Delta t = 0.05$ s, $K = 1000$, $\lambda = 0.3$ and $\Sigma^{1/2} = \text{diag}(0.2, 0.4)$ (Table 5).

3.2. Detecting Entrapment: A Single Monotone Progress Signal

The nearest-path index $k_t = \arg \min_i \|\mathbf{p}_t - \sigma_i\|$ is non-monotone: it jumps back when the robot backs off a wall. We therefore track the furthest index ever reached and count the cycles in which it fails to advance,

$$\begin{aligned} k_t^{\max} &= \max(k_{t-1}^{\max}, k_t), \quad s_t = \begin{cases} 0 & k_t^{\max} > k_{t-1}^{\max}, \\ s_{t-1} + 1 & \text{otherwise,} \end{cases} \\ e_t &= \begin{cases} 0 & k_t^{\max} > k_{t-1}^{\max} \text{ or } \|\mathbf{p}_t - \sigma_M\| \leq \epsilon_{\text{goal}}, \\ 1 & s_t \geq W, \\ e_{t-1} & \text{otherwise.} \end{cases} \end{aligned} \quad (3)$$

Entrapment thus latches after W stalled cycles ($W = 30$ by default) and clears on the first real progress, so stop-and-go motion during an escape cannot leave the flag stuck; within ϵ_{goal} of the path end the

detector is reset, so a robot finishing at the goal does not trigger a false escape. Within SE-MPPI, the controller’s detector is the single entrapment source: the critic and the coordinator read the same atomic flag through a registry keyed by the controller name, so escape and safety always agree on whether the robot is trapped. (When the critic runs under a stock MPPI controller, it falls back to its own detector on MPPI’s furthest-reached index.)

3.3. Escaping Local Minima: Gated Repulsion and Gap Attraction

Only while $e_t = 1$, the EscapeCritic adds two terms to every rollout’s cost in (2); in free space the controller is therefore exactly stock MPPI (detect-and-switch rather than always-on, as in DRPA-MPPI (Fuke et al., 2025a)). The first is a distance-field artificial potential of the classical repulsive form (Khatib, 1986). Let $d(\mathbf{q})$ be the 8-connected Dijkstra distance from costmap cell $\mathbf{q}$ to the nearest obstacle cell, where unknown cells are *not* obstacles so that unexplored space does not repel the escape. With $\mathcal{J}^k$ the on-map poses of rollout k and $d_{\min}$ half a cell,

$$U(d) = \begin{cases} \frac{1}{2} \eta \left(\frac{1}{\max(d, d_{\min})} - \frac{1}{d_0} \right)^2 & d < d_0, \\ 0 & d \geq d_0, \end{cases} \quad C_{\text{apf}}(\mathbf{x}_{0:T}^k) = \left(\frac{1}{|\mathcal{J}^k|} \sum_{j \in \mathcal{J}^k} U(d(\mathbf{p}_j^k)) \right)^p, \quad (4)$$

which pushes rollouts off the trapping surface. Repulsion alone does not route the robot *around* a non-convex trap, so the second term supplies a temporary heading toward an opening (a bearing cost; the goal itself is unchanged), a lightweight form of the free-space subgoal and gap guidance of Mohamed et al. (2023) and Xu et al. (2021). The critic casts N_r rays at bearings $\psi_i = -\pi + 2\pi i/N_r$ from the robot, measures each ray’s free length $\ell(\psi_i) \leq r_{\max}$ on the costmap, and selects the open bearing closest to the goal bearing ψ_{goal} ; rollouts whose endpoint heads toward it are rewarded:

$$\psi^* = \arg \min_{\psi_i: \ell(\psi_i) \geq r_{\min}} |\text{wrap}(\psi_i - \psi_{\text{goal}})|, \quad C_{\text{gap}}(\mathbf{x}_{0:T}^k) = w_g \left(1 - \cos \text{wrap}(\angle(\mathbf{p}_T^k - \mathbf{p}_t) - \psi^*) \right)^{p_g}. \quad (5)$$

The defaults are $d_0 = 0.6$ m, $\eta = 1$, $N_r = 36$, $r_{\max} = 2$ m, $r_{\min} = 0.6$ m and $w_g = 4$ (Table 5). In the 2D mirror of Section 4, which uses the selected bearing as a temporary subgoal rather than through C_{gap} (Section C), disabling the gap search removes every escape (Section 4.3); the C_{gap} term of the C++ critic has not been ablated on its own.

3.4. Tracking Dynamic Obstacles: Costmap Clusters with Gated Association

The tracker clusters local-costmap cells at or above the inscribed-inflation threshold, summarizing each cluster c by a centroid $\mathbf{c}$ and a radius. Each current cluster is associated with at most one previous track within a gate of 0.6 m, each track being consumable once, so a split or merge cannot manufacture a phantom velocity. The velocity is a clamped finite difference over the track history,

$$\mathbf{v}_o = \text{clamp}_{v_{\text{track}}} \left(\frac{\mathbf{c}_t - \mathbf{c}_{t'}}{t - t'} \right), \quad \mathcal{O}_t = \{o : \|\mathbf{v}_o\| \geq v_{\text{dyn}}, R_o \leq R_{\max}\}, \quad (6)$$

with t' the previous observation of the track and $v_{\text{track}} = 2$ m/s. Only obstacles that are moving and small enough to be a movable body enter $\mathcal{O}_t$, and hence the coordinator and the CBF. Static structure is deliberately excluded and left to the MPPI obstacle critic and costmap inflation; Section 4.4 shows why this scoping is essential on a real costmap. The constant-velocity model is a replaceable baseline (Section 5).

3.5. Filtering Commands: A Look-Ahead-Point CBF Quadratic Program

A distance barrier on the unicycle's centre has relative degree two in ω . We therefore enforce safety on the look-ahead point $\mathbf{P}_t = \mathbf{p}_t + L(\cos \theta_t, \sin \theta_t)$, which (v, ω) actuates fully:

$$\dot{\mathbf{P}}_t = G(\theta_t) \mathbf{u}_t, \quad G(\theta) = \begin{bmatrix} \cos \theta & -L \sin \theta \\ \sin \theta & L \cos \theta \end{bmatrix}. \quad (7)$$

For each $o \in O_t$, with $\mathbf{d}_o = \mathbf{P}_t - \mathbf{p}_o$ and effective radius $R_o^{\text{eff}} = r + R_o + m$, the barrier and its derivative are

$$h_o = \|\mathbf{d}_o\|^2 - (R_o^{\text{eff}})^2, \quad \dot{h}_o = 2 \mathbf{d}_o^\top (G(\theta_t) \mathbf{u}_t - \mathbf{v}_o), \quad (8)$$

and the continuous-time CBF condition (Ames et al., 2017, 2019)

$$\dot{h}_o + \alpha_t h_o \geq 0 \quad (9)$$

is linear in $\mathbf{u}_t$. Once per control cycle we solve, with OSQP (Stellato et al., 2020),

$$\min_{v, \omega, \delta \geq 0} w_v (v - v_t^{\text{nom}})^2 + w_\omega (\omega - \omega_t^{\text{nom}})^2 + \rho \delta^2 \quad \text{s.t.} \quad 2 \mathbf{d}_o^\top G(\theta_t) \mathbf{u} + \delta \geq -\alpha_t h_o + 2 \mathbf{d}_o^\top \mathbf{v}_o \quad \forall o, \quad \mathbf{u} \in \mathcal{U}, \quad (10)$$

where the nominal command is first clamped to $\mathcal{U}$ and a single shared slack δ keeps the QP feasible. Obstacles are pruned to the N_{obs} nearest by *clearance* (surface gap) rather than centre distance, so a large obstacle is never dropped in favour of a smaller one whose centre is closer. If the QP can stay safe only by relaxing the barrier ($\delta > \epsilon_\delta = 10^{-3}$) or fails, the controller brakes, setting $v = 0$ while keeping the QP's turn (or, on solver failure, the clamped nominal turn): it stops rather than drive into an imminent collision.

Because (9) is imposed only at sampling instants, strict between-sample invariance would require tightening R_o^{eff} by a term of order $\dot{h}_{\text{max}} \Delta t$. In practice the margin m absorbs this gap, and we report the empirical slack usage instead of claiming continuous-time invariance between samples.

3.6. Coordinating Escape and Safety: Gain Scheduling under a Conditional Invariance Lemma

The coordinator resolves the barrier gain from the entrapment flag and the minimum time-to-collision under the current command,

$$\alpha_t = \begin{cases} \alpha_{\text{escape}} & e_t = 1 \text{ and } \text{TTC}_t \geq \tau, \\ \alpha_{\text{base}} & \text{otherwise,} \end{cases} \quad \text{TTC}_t = \min_{o \in O_t: \kappa_o > 0} \frac{\|\mathbf{r}_o\| - (r + R_o)}{\kappa_o}, \quad \kappa_o = -\frac{\mathbf{r}_o^\top (\mathbf{v}_o - v_t^{\text{nom}} \mathbf{h}_t)}{\|\mathbf{r}_o\|}, \quad (11)$$

where $\mathbf{r}_o = \mathbf{p}_o - \mathbf{p}_t$, $\mathbf{h}_t = (\cos \theta_t, \sin \theta_t)$, κ_o is the closing speed, $\text{TTC}_t = 0$ on contact and $+\infty$ if nothing approaches, and $\alpha_{\text{escape}} > \alpha_{\text{base}} > 0$ (defaults 6 and 2, $\tau = 1.5$ s).¹ Raising α_t relaxes the lower bound $\dot{h}_o \geq -\alpha_t h_o$, letting the robot approach an obstacle faster as an escape requires, while the TTC override snaps the gain back to α_{base} when a dynamic obstacle is imminent. The next result states that, under explicit conditions, this schedule does not by itself give up forward invariance. It is a known property of CBFs with time-varying class- $\mathcal{K}$ terms and follows the same argument as the relative-degree-one case of the invariance result for adaptive CBFs with time-varying penalty functions (Xiao et al., 2020, Thm. 3), which is stated there for Lipschitz-continuous controllers, whereas our switched gain yields

¹The released controller also holds α_t at α_{base} while an online conformal bound on obstacle-prediction error exceeds a trust threshold, and that bound inflates R_o^{eff} . This is the follow-up conformal margin of Section 2, not claimed here; an implementation of (11) alone reproduces the benchmark configurations of Section 4, not the released defaults.

a discontinuous command; related time-varying and optimized decay rates appear in Zeng et al. (2021) and Parwana and Panagou (2025). We restate it with the standard integrating-factor proof for completeness and claim no new mathematics.

Assumption 1 (Slack-free barrier between samples). For a fixed obstacle j and all $t \geq 0$ in the interval considered, the QP (10) is feasible with $\delta(t) = 0$, and the continuous-time inequality $\dot{h}_j(t) \geq -\alpha(t) h_j(t)$ holds for almost every t .

The second clause is an assumption, not a consequence: feasibility at discrete instants does not by itself imply the inequality between samples (the sampled-data caveat of Section 3.5).

Proposition 1 (Invariance under a time-varying gain; after Xiao et al., 2020). *Let $\alpha : [0, \infty) \rightarrow [\alpha_{\min}, \alpha_{\max}] \subset (0, \infty)$ be any measurable, positive, bounded gain schedule, in particular the piecewise-constant switching (11) with its TTC override. Under Assumption 1, if $h_j(0) \geq 0$ then $h_j(t) \geq 0$ for all $t \geq 0$: the look-ahead point does not enter obstacle j 's inflated disc, as modelled.*

Proof. Define $\varphi(t) = \exp(\int_0^t \alpha(s) ds)$, which is finite and strictly positive because $0 < \alpha \leq \alpha_{\max}$. Almost everywhere,

$$\frac{d}{dt} [\varphi(t) h_j(t)] = \varphi(t) [\dot{h}_j(t) + \alpha(t) h_j(t)] \geq 0, \quad (12)$$

so φh_j is non-decreasing and $\varphi(t) h_j(t) \geq \varphi(0) h_j(0) = h_j(0) \geq 0$. Since $\varphi(t) > 0$, $h_j(t) \geq 0$ for all $t \geq 0$. $\square$

Positivity and boundedness are stronger than the argument needs: a nonnegative, locally integrable α already makes φ finite and positive. We keep the stronger hypothesis because it matches the schedule (11).

Corollary 1. *Under Assumption 1, forward invariance of $C_j = \{h_j \geq 0\}$ holds for every positive bounded schedule, including the switch to α_{escape} and the TTC override back to α_{base} . If Assumption 1 holds for every row, $C = \bigcap_j C_j$ is invariant; the argument is per obstacle and gain-agnostic, so sharing one α_t across rows does not affect it.*

Under Assumption 1, raising α therefore does not permit $h_j < 0$; it only weakens the lower bound on $\dot{h}_j$, admitting bolder detours inside the same set. This is the design rationale for coordinating rather than stacking the two layers: in this idealized setting, the gain trades caution for manoeuvrability without giving up invariance. The guarantee is narrow. It concerns the look-ahead point, not the robot body; it covers only the tracked dynamic obstacles under the constant-velocity model, not static structure; it applies to an obstacle only while that obstacle stays in the tracked, unpruned set O_t (an obstacle that slows below v_{dyn} , merges into a cluster larger than R_{max} , or is pruned beyond the N_{obs} smallest clearances leaves the QP); it assumes the continuous-time inequality between samples; and it requires $\delta = 0$, so it does not cover any cycle in which the QP uses slack. Because the slack is shared, one imminent obstacle can momentarily loosen the margin on every other obstacle in the same solve, so we report slack usage empirically: in the benchmark of Section 4, the QP never uses slack on the mover-free families, and uses nonzero slack in 26–34 of 50 narrow-dynamic trials (maximum 0.47) and in 47–48 of 50 dynamic-family trials (maximum 0.015) per CBF-carrying configuration (Table 7). In cycles with $\delta > \epsilon_\delta$ the brake of Section 3.5 replaces the QP command.

Algorithm 1: One SE-MPPI control cycle (computeVelocityCommands)

Input: pose $\mathbf{x}_t$, global path σ , local costmap, previous state $(k_{t-1}^{\max}, s_{t-1}, e_{t-1})$
Output: safe command $\mathbf{u}_t^{\text{safe}}$

- 1 $\mathbf{u}_t^{\text{nom}} \leftarrow$ stock MPPI optimizer, scoring rollouts with (2); the EscapeCritic adds $C_{\text{apf}} + C_{\text{gap}}$ iff the shared flag $e_{t-1} = 1$
- 2 $k_t \leftarrow \arg \min_i \|\mathbf{p}_t - \sigma_i\|$; update $k_t^{\max}, s_t, e_t$ by (3); publish e_t to the shared registry
- 3 $O_t \leftarrow$ tracked clusters that are moving and small (6)
- 4 $\alpha_t \leftarrow$ gain from e_t and TTC_t by (11)
- 5 **if** $O_t = \emptyset$ **then**
- 6 $\mathbf{u}_t^{\text{safe}} \leftarrow \text{clip}_{\mathcal{U}}(\mathbf{u}_t^{\text{nom}})$
- 7 **else**
- 8 keep the N_{obs} obstacles of smallest clearance; solve the QP (10) with gain α_t
- 9 **if** *solver failed* **or** $\delta > \epsilon_\delta$ **then**
- 10 $v_t^{\text{safe}} \leftarrow 0$ // brake, keep the turn
- 11 **return** $\mathbf{u}_t^{\text{safe}}$

3.7. Packaging for Nav2: Two Plugins over a Plugin-Free Core

SE-MPPI is built on ROS 2 Jazzy and Nav2 (Macenski et al., 2020). The package `nav2_se_controller` ships three shared libraries: a plugin-free `se_mppi_core` (entrapment, CBF filter, coordinator, tracker, repulsion, gap search), the `escape_critic` MPPI critic, and the `safe_escape_controller`. Separating the core from the two plugin libraries resolves a `class_loader` factory conflict that otherwise prevents the critic and the controller from loading together. Two integration details are easy to get wrong. First, Nav2’s `CriticManager` resolves a critics-list entry NAME to the class `mppi::critics::NAME`; a critic in any other namespace passes a direct pluginlib unit test yet never loads from the critics list, so `EscapeCritic` lives in `mppi::critics`. Second, the Jazzy critic data are `xtensor` tensors, not the Eigen types of Nav2’s development branch, so the critic is written against the installed headers. The QP uses `osqp-eigen` (Stellato et al., 2020), and the build is reproducible through RoboStack (conda-forge) independently of the host OS. The modules this paper relies on (entrapment detector, CBF filter, coordinator, tracker, gap search, repulsion, path progress and plugin loading) carry 44 gtest cases; with the follow-up prediction and multi-robot modules and the live-injection regression of Section 4.4, the algorithm test suite (`test/test_*.cpp`) holds 82 TEST cases in 13 files. The code described and evaluated in this paper is the release tagged `paper-v1`; later commits add deployment tests and fixes that are not part of the evaluated code.

4. Experiments

We first check each mechanism in isolation, then measure the contribution of every layer on a randomized paired benchmark run on a Python 2D mirror of the controller (Section 4.2), isolate the gap search, the coordination and the CBF by ablation (Section 4.3), and finally report what a live Nav2 deployment confirms and exposes (Section 4.4).

4.1. Experimental Setup

Evaluation Surface. Every quantitative result in this section comes from a Python 2D mirror, not from the Nav2 controller. The mirror is a randomized 2D benchmark that re-implements the controller’s

primitives in NumPy and OSQP: the MPPI sampler, the APF repulsion, the gap raycast, the look-ahead-point CBF-QP, the α coordination with TTC override, and the monotone-progress entrapment detector. As in the released controller and the deployment lesson of Section 4.4, the CBF and the TTC see only the dynamic obstacles, never static walls. Like the controller, the mirror enforces the barrier on the look-ahead point rather than the full body; unlike it, the mirror uses distance-to-goal progress and a free-space gap subgoal in place of a global path, and computes geometry analytically against circular obstacles instead of on a costmap; Section C lists every difference. Two differences matter most for reading the results: the mirror uses the selected gap as a temporary subgoal that replaces the goal, whereas the C++ critic keeps the goal and the path-following critics and adds the bearing cost C_{gap} of (5) toward the selected gap, and the mirror gives the CBF and the TTC the true obstacle states, so the costmap tracker of Section 3.4 is not exercised. We use this surface instead of a live Gazebo matrix because on the only host available to us (a WSL2 workstation running at a real-time factor of about 0.3–0.6 under CPU contention, with transform smear adding phantom cells to the costmap) a live comparison measures the host rather than the algorithm.

Scenario Families. Four seed-deterministic families exercise the two weaknesses and their conjunction. *U-trap*: a finite U- or C-shaped pocket with the goal behind its back wall, a static local minimum. *Clutter*: a random field of 5–9 circular obstacles with a guaranteed footprint-clear path. *Dynamic*: one or two pedestrian-like movers timed to cross the robot’s path. *Narrow-dynamic*: the U-trap pocket plus movers sweeping its mouth during the escape window, the narrow-and-dynamic coincidence the coordination targets.

Configurations. Six ablation configurations toggle the mechanisms: **A** stock MPPI (no escape, no CBF); **C** escape only; **D** CBF only; **E** escape + CBF with $\alpha_{\text{escape}} = \alpha_{\text{base}}$, i.e. independent layers; **F** full SE-MPPI with $\alpha : 2 \rightarrow 6$ and TTC override, i.e. coordinated; and **F⁻** F with the gap search disabled. Configuration A is stock MPPI without Nav2’s progress checker and behaviour-tree recoveries (Nav2 contributors, 2026b,c), so it is a controller-level baseline, not the full Nav2 stack. The deployable baselines of the specified full-scale protocol (DWB, RPP, TEB, an always-on variant of our escape critic and a Shield-style CBF-only controller) have not been run, nor have DRPA-MPPI itself (Fuke et al., 2025a), GP-MPPI (Mohamed et al., 2023), a gap-based planner (Feng et al., 2023) or an adaptive-gain CBF (Kim et al., 2025); we make no comparison to them (Section 5).

Protocol and Metrics. Every scenario is a pure function of an integer seed, and the MPPI noise stream uses the same seed, so all configurations face byte-identical worlds and rollout noise, as the paired design requires. We run 4 families $\times$ 50 seeds $\times$ 6 configurations = 1,200 trials with a budget of 400 steps of 0.1 s; these are 200 seeded worlds, each run under all six configurations, so the trials are paired rather than independent. A trial is a success if it reaches the goal without a footprint collision, a collision if the footprint overlaps an obstacle, and a timeout otherwise. Success carries a Wilson 95% interval (Wilson, 1927); the primary test is McNemar’s (McNemar, 1947) on paired outcomes; continuous metrics (time-to-goal, path length, clearance) use the Mann–Whitney U test (Mann and Whitney, 1947) with Cliff’s δ (Cliff, 1993), an unpaired comparison that ignores the pairing, with time-to-goal and path length taken over successful trials only, so we read these as descriptive; all p -values are Holm-corrected (Holm, 1979) within each family (16 tests per family). The benchmark runs on CPU in Python; we report no per-cycle compute figures (Section 5). Parameters are in Table 5, and commands in Section A.

4.2. Main Results

Mechanism Check. Before measuring rates, we verify that each mechanism does what it is designed to do on three hand-built scenes (Figure 2, Table 2). Stock MPPI stalls in front of a finite wall for

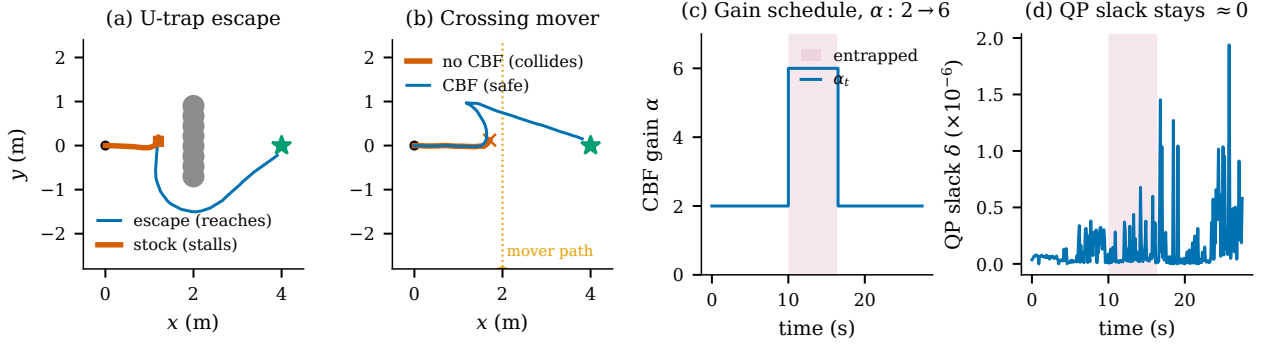


Figure 2: Mechanism validation in the 2D mirror (seed-fixed single runs; Table 2). (a) Stock MPPI stalls at the mouth of a finite wall ($x \approx 1.2$ m) while the detection-and-escape layer rounds it to the goal. (b) Against a mover crossing the path, the no-CBF controller collides ($\times$) while the look-ahead-point CBF filter keeps the robot clear. (c, d) On the U-trap, the coordinated gain rises from $\alpha_{\text{base}}=2$ to $\alpha_{\text{escape}}=6$ while entrapped (shaded) and the QP slack stays below 2×10^{-6} throughout: the escape is admitted without relaxing the barrier, close to the slack-free setting that Proposition 1 assumes. Regenerated as vector graphics by `scripts/make_figures.py`.

Table 2: Mechanism validation (single seed-fixed runs of the 2D mirror; 40 s budget). “Stalled” = budget exhausted without progress. Source: `data/mechanism_validation.json`, regenerated from `experiments/prototype/run_validation.py`.

Scenario	Configuration	Reached	Collided	Time (s)	Min. clear. (m)	$\max_t \alpha_t$
U-trap	stock MPPI (A)	no (stalled)	no	–	0.33	2
U-trap	escape (C)	yes	no	27.7	0.32	2
Crossing mover	escape, no CBF (C)	no	yes	–	–0.00	2
Crossing mover	SE-MPPI (F)	yes	no	18.6	0.01	6
U-trap	SE-MPPI (F)	yes	no	27.7	0.32	6
U-trap + mover	independent (E)	yes	no	27.7	0.32	2

the full 40 s budget, while the escape layer reaches the goal in 27.7 s; against a crossing mover the no-CBF controller collides after 4.7 s, while the CBF filter reaches the goal in 18.6 s with the minimum clearance never negative; and on the U-trap the coordinated gain rises from 2 to 6 while entrapped, with QP slack below 2×10^{-6} throughout, so the escape is admitted without relaxing the barrier, close to the slack-free setting that Proposition 1 assumes. In these benign scenes the coordinated and independent variants are both safe and equally fast (27.7 s, 0.32 m clearance), so they cannot separate the two; the randomized benchmark below is the quantitative test.

The Escape Layer Is the Measured Contribution in the Mirror. On both trap families the effect is decisive and survives Holm correction (Tables 3 and 4). Success rises from 0%, where stock A, CBF-only D and no-gap F^- all time out in the pocket, to 88–90% on U-trap (44 of 50 discordant pairs in F ’s favour, $p_{\text{adj}} = 1.4 \times 10^{-9}$) and to 62–78% on narrow-dynamic (31 of 50, $p_{\text{adj}} = 1.1 \times 10^{-6}$). On clutter the escape adds 10 percentage points (52% $\rightarrow$ 62%, not significant after correction) at the cost of about 2.4 s longer mean time-to-goal over successful trials (15.2 s $\rightarrow$ 17.7 s; Table 6). On the pure dynamic family, which contains no trap, it is appropriately neutral (A $\rightarrow$ F: $b/c = 3/2$, $p_{\text{adj}} = 1$). Figure 3 shows the full per-family picture, including collision and timeout rates.

4.3. Ablation Studies

The Gap Search Is Load-Bearing. F^- , which keeps the entrapment detector, the APF repulsion and the gain modulation but drops the gap search, scores 0% on both trap families (Table 3), identical

Table 3: Main result: success rate in % with Wilson 95% confidence interval, $N = 50$ paired scenarios per cell (1,200 trials of the Python 2D mirror, not the Nav2 controller). Best per family in bold. On the two trap families the escape configurations (C, E, F) separate from stock (A), CBF only (D) and the no-gap ablation (F^-), which all stay at 0%. Source: `experiments/results_2d/summary.csv`.

Configuration	U-trap	Clutter	Dynamic	Narrow-dyn.
A stock MPPI	0 [0,7]	52 [39,65]	80 [67,89]	0 [0,7]
C escape only	90 [79,96]	62 [48,74]	76 [63,86]	78 [65,87]
D CBF only	0 [0,7]	52 [39,65]	82 [69,90]	0 [0,7]
E escape+CBF (indep.)	88 [76,94]	62 [48,74]	78 [65,87]	64 [50,76]
F SE-MPPI (coord.)	88 [76,94]	62 [48,74]	78 [65,87]	62 [48,74]
F^- F without gap search	0 [0,7]	52 [39,65]	84 [71,92]	0 [0,7]

Table 4: Paired contrasts against F (McNemar on success, Holm-adjusted within each family of 16 tests), Python 2D mirror. b = trials the baseline solves and F does not; c = the reverse. * marks $p_{\text{adj}} < 0.05$. The escape contrast ($A \rightarrow F$) is decisive on both trap families; the coordination contrast ($E \rightarrow F$) is null everywhere. Source: `experiments/results_2d/stats.json`.

Family	Escape ($A \rightarrow F$)		CBF cost ($C \rightarrow F$)		Coordination ($E \rightarrow F$)	
	b/c	p_{adj}	b/c	p_{adj}	b/c	p_{adj}
U-trap	0/44	$1.4 \times 10^{-9*}$	1/0	1	0/0	1
Clutter	0/5	1	0/0	1	0/0	1
Dynamic	3/2	1	1/2	1	0/0	1
Narrow-dyn.	0/31	$1.1 \times 10^{-6*}$	8/0	0.109	1/0	1

to stock MPPI. In the mirror, detection-and-switch escapes only *with* the free-space gap subgoal: repulsion and gain modulation without a routed opening do not escape. There the gap acts as a temporary subgoal (Section C), a form close to the subgoal guidance of GP-MPPI (Mohamed et al., 2023); whether the C++ bearing cost C_{gap} is equally load-bearing has not been tested.

Escape–Safety Coordination Is Null in This Regime. The contrast that bears on the coordination contribution is E (independent) versus F (coordinated): identical mechanisms that differ only in whether the CBF gain follows the escape phase. Across all 200 paired scenarios this contrast is null (Table 4, Figure 4): the discordant counts are 0/0, 0/0, 0/0 and 1/0, McNemar $p = 1$ in every family, and every continuous effect size (Cliff’s δ) satisfies $|\delta| \leq 0.032$. The gain does engage as designed ($\max_t \alpha_t = 6$ during escape, TTC override active). The per-trial slack telemetry (Table 7) locates where the barrier actually works. On the mover-free families the dynamic-obstacle CBF sees no obstacle at all: slack is zero in every trial, E and F issue identical commands, and the null there is structural. On narrow-dynamic the barrier *does* engage and even relaxes, with nonzero slack in 34 of 50 trials for E and F (26 for D and F^- , maximum 0.47), and on dynamic in 47–48 of 50 (maximum 0.015); yet E and F still do not differ. The correct reading is therefore not that the barrier never binds, but that the *outcomes are insensitive to the α schedule*: where the barrier is quiescent the two configurations coincide exactly, and where it is active both gains admit escapes whose outcomes are indistinguishable at $N = 50$. In this benchmark the coordination produces no measurable outcome difference over independent escape+CBF. This is consistent with, and delimits, Proposition 1: coordination is a mechanism with a conditional invariance property, not, in these scenarios, an outcome improvement.

What the CBF Changes. Against body-grazing crossers (dynamic) the look-ahead-point filter leaves the collision rate essentially unchanged (20% for A versus 16–18% with a CBF; descriptive, since the committed analysis carries no paired collision test). In the cramped narrow-dynamic pocket, the CBF configurations show a *higher* collision rate than their no-CBF counterparts (18% $\rightarrow$ 32%,

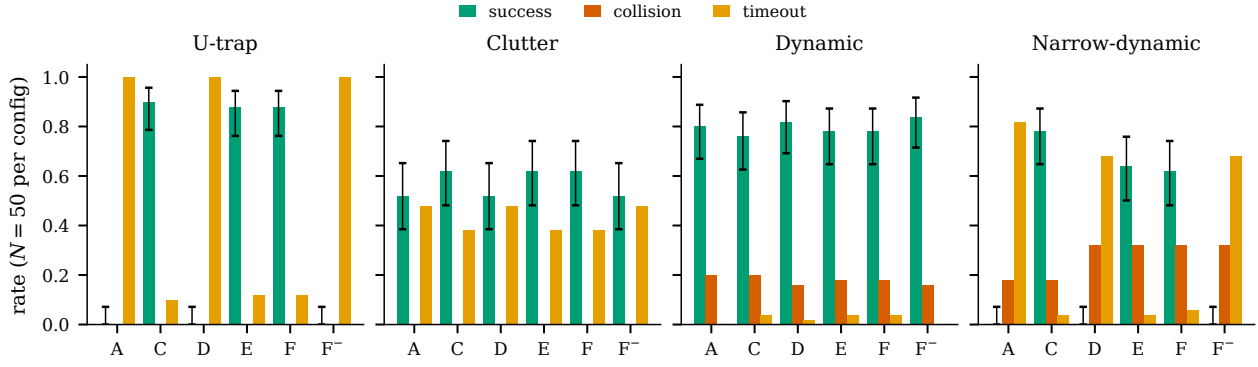


Figure 3: Outcome rates per family in the Python 2D mirror across configurations A–F[−] ($N = 50$ each; Wilson 95% intervals on success). On the trap families (U-trap, narrow-dynamic) the escape configurations C, E and F separate from A, D and F[−], which time out at 0% success. Plotted directly from `experiments/results_2d/summary.csv`.

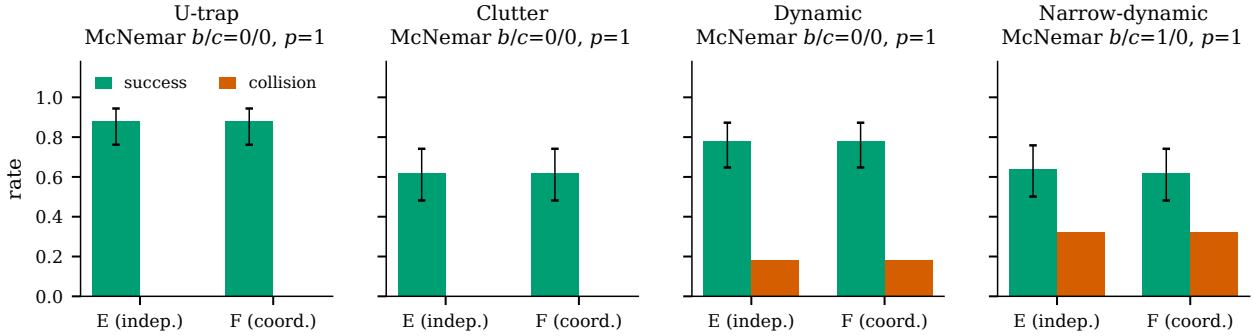


Figure 4: The coordination contrast is null. Independent escape+CBF (E) versus coordinated SE-MPPI (F) per family in the Python 2D mirror, with the paired McNemar discordant counts and raw p -values from `experiments/results_2d/stats.json`. The two are indistinguishable in every family.

descriptive) and lose about 16 points of success (C 78% versus F 62%; McNemar $b/c = 8/0$, raw $p = 0.008$, $p_{\text{adj}} = 0.109$), which is directionally consistent though not significant after correction at $N = 50$. We attribute both to the braking fallback and to the barrier acting on a single look-ahead point, and discuss them as limitations in Section 5.

4.4. Live Nav2 and Gazebo Integration

We deployed SE-MPPI in a complete live stack: ROS 2 Jazzy, Nav2 (AMCL, costmaps, NavFn, behaviour-tree navigator, recovery, collision monitor) and the Gazebo `tb3_sandbox` world. The committed launch logs and trial telemetry confirm that the controller and the `EscapeCritic` load and activate from the critics list, that the controller produces velocity commands of the same form as stock MPPI, and that the lifecycle reaches the active state and sustains multi-minute drives on a U-trap testbed without a crash or collision after the fix below. The goal was not reached: the committed telemetry records STUCK at 254.5 s and TIMEOUT at 299.8 s, both collision-free, under the host constraints of Section 4.1. The escape-injection path has also run live: its first live activation exposed the ABI fault below, and the tensor shapes captured from that event are committed as a regression test, while the controller and critic now log every detection and injection transition for auditability. No end-to-end Nav2 escape has therefore been demonstrated: a sustained live entrapment-and-escape run remains confounded by the host artefacts and is deferred to a GPU workstation. Three faults surfaced that

only a live stack reveals:

- *Static walls in the CBF.* Feeding every lethal cluster, walls included, into the CBF makes the barrier infeasible everywhere and freezes the robot; scoping the CBF to moving, body-sized obstacles (6) fixes it.
- *Compute budget.* The per-cycle budget must match the host (horizon preserved, rate and batch reduced), or the loop misses its rate and the optimizer diverges.
- *xtensor ABI mismatch.* The installed `libmppi_controller.so` is compiled with `XTENSOR_USE_XSIMD` and `AVX2` (an aligned-allocator element type), whereas our plugin used the default allocator. Because the critic tensors cross the pluginlib boundary by reference, the first `EscapeCritic` write to `data.costs` violated the one-definition rule and crashed the whole Nav2 process on the first live escape injection. Unit tests could not catch it, since they compile every translation unit with one flag set, and earlier live runs had never triggered entrapment. Compiling the critic with the installed headers' flags, per target because a global `AVX2` switch breaks the separately built `OSQP-Eigen` boundary, fixes it, and a regression test replays the injection with the live tensor shapes.

This is the class of fault a Nav2-native deployment exposes and that a standalone node or a pure 2D study cannot.

5. Discussion and Limitations

Where Should Coordination Matter? The null E-versus-F result does not contradict the design; it bounds where the design can pay off. An outcome-level benefit requires a regime in which the choice of α changes which escape manoeuvres are admitted at outcome-deciding moments: faster or persistent movers, tighter margins, or a dynamic gap the escape route must thread while the barrier is active. Our generator's pedestrian-like movers produce barrier engagement (Table 7) but not that sensitivity. Constructing and measuring such a regime is future work, and we claim no coordination benefit we have not measured.

Cost Analysis. We do not report per-cycle compute. The benchmark runs a Python mirror whose timings say nothing about the C++ plugin, and the compute telemetry of the committed live pilot trials is empty ($n = 0$ samples), so any figure would be unsupported. The QP adds one small solve per cycle whose size grows with the obstacle budget N_{obs} (capped by clearance pruning), and the escape terms are evaluated only while entrapped; measuring their cost on a real-time-capable host belongs to the full-scale benchmark.

Deployment Lessons. The three live faults of Section 4.4 share a pattern: each sits on an interface (costmap semantics, the compute budget, the binary interface across the pluginlib boundary) that unit tests with one consistent build configuration and synthetic inputs do not exercise. For controller plugins that write into another library's tensors, we recommend compiling against the installed headers' exact flags and committing a regression test that replays live-captured shapes.

Limitations. We state the limits of the evidence explicitly.

- *Full-scale physics benchmark and baselines not run.* The quantitative comparison is the 2D mirror, not the live controller. The specified full-scale protocol, an identical Nav2 stack and differential robot across BARN (Perille et al., 2020; Xiao et al., 2024), DynaBARN (Nair et al., 2022) and HuNavSim (Pérez-Higueras et al., 2023) against stock MPPI (Macenski et al., 2020; Williams et al., 2018), DWB (Fox et al., 1997), RPP (Macenski et al., 2023), TEB (Rösmann

et al., 2012), an always-on variant of our escape critic and a Shield-style CBF-only controller (Yin et al., 2023), scored by success, collision, time-to-goal, path length, clearance and compute under the same statistics, needs a GPU workstation we did not have. The live entrapment-and-escape reproduction is deferred for the same reason; no live run has yet reached its goal, so controller-level escape in Nav2 is not demonstrated. The planned evaluation also adds the baselines that the prior work of Section 2 calls for: stock MPPI with Nav2’s progress checker and behaviour-tree recoveries enabled (Nav2 contributors, 2026b,c), DRPA-MPPI (Fuke et al., 2025a), a GP-MPPI-style subgoal escape (Mohamed et al., 2023), a gap-based planner (Feng et al., 2023) and an adaptive-gain CBF (Kim et al., 2025).

- *Mirror versus controller.* Every quantitative result comes from the Python 2D mirror, whose progress signal, gap use, geometry, obstacle knowledge and costs differ from the C++ controller (Section C). In particular, the load-bearing gap result concerns the mirror’s temporary subgoal, not the critic’s C_{gap} term, and the costmap tracker is not exercised.
- *Statistics.* The 1,200 trials are 200 seeded worlds under six configurations, not independent scenarios. Success is tested with paired McNemar tests, but the continuous metrics use unpaired Mann–Whitney tests, with time-to-goal and path length conditioned on success, so they are descriptive; a paired analysis that handles failures explicitly belongs to the full-scale benchmark.
- *Coordination benefit unmeasured.* The coordination has the conditional invariance property of Proposition 1 but produced no outcome difference over independent escape+CBF (Section 4.3), and the idea of adapting the CBF gain online is not new (Kim et al., 2025; Parwana and Panagou, 2025; Parwana et al., 2025).
- *Braking is not a safe fallback when cornered.* In the narrow-dynamic pocket the infeasible-QP fallback ($v = 0$) leaves the robot in the path of a sweeping mover, and the CBF configurations collide more often than their no-CBF counterparts (18% $\rightarrow$ 32%). A cornered robot needs an evasive fallback, not a stop.
- *Look-ahead point versus full body.* Proposition 1 concerns the look-ahead point $\mathbf{P}_t$, not the robot body. Body safety is approximate: $L \approx r$ places $\mathbf{P}_t$ at the leading edge, and the costmap critic and `collision_monitor` cover the residual. A full-body guarantee would inflate the margin by L , which we avoid because it over-constrains sub-metre gaps. The gap is visible in the data: body-grazing crossers still collide at 16–18% with the filter.
- *Sampled-data and slack caveats.* Invariance between samples is assumed (Assumption 1), not proven, and the shared slack couples the rows of one solve; we report slack usage instead of claiming an unconditional guarantee. Static structure is outside the barrier altogether, and in cycles with $\delta > \epsilon_\delta$ the braking fallback replaces the QP command.
- *Constant-velocity tracker.* The obstacle model evaluated here is a constant-velocity baseline that miscalculates turning or accelerating agents and does not quantify its uncertainty, and an obstacle that leaves the tracked, unpruned set $\mathcal{O}_t$ also leaves the barrier (Section 3.6). The released controller has since added occupancy-persistence static/dynamic classification, least-squares horizon prediction and an online conformal radius; these belong to follow-up work, and none of them is present in the 2D mirror behind every quantitative result reported here.
- *Other approximations.* The TTC is a one-dimensional closing-speed estimate that can induce over-rotation in tight spaces, and localization jumps in symmetric or narrow maps can perturb the progress signal.

6. Conclusion

We presented SE-MPPI, a system that packages local-minima escape and control-barrier-function safety, two problems that several prior works also address together outside Nav2, in a single Nav2 controller plugin: one entrapment signal drives both a sampling-time escape critic and an output-time CBF filter, and a coordinated gain raises the barrier’s class- $\mathcal{K}$ gain during escape, which, by a standard argument for time-varying gains, keeps the look-ahead point’s safe set with respect to the tracked dynamic obstacles forward invariant, assuming the barrier inequality holds between control updates and the QP uses no slack. On a randomized, paired 1,200-trial benchmark run on a Python 2D mirror of the controller’s primitives, the online detection-and-escape layer is the decisive contribution, raising success from 0% to 88–90% on the U-trap family and from 0% to 62–78% on the narrow-dynamic family, with the mirror’s free-space gap subgoal load-bearing, whereas the coordination is statistically indistinguishable from independent escape+CBF in the regimes we could test. The controller loads, activates and runs in a full ROS 2 Jazzy, Nav2 and Gazebo stack, where three integration faults surfaced that only a live deployment reveals, although no live run has yet reached its goal. The remaining steps are the full-scale physics benchmark on a real-time-capable GPU host, with Nav2’s own stall recovery, DRPA-MPPI, subgoal and gap-based baselines, and a successful end-to-end Nav2 escape.

References

- ACCESS Newswire. Open Navigation and 3Laws Demonstrate Capabilities of Nav2 and 3Laws Supervisor Integration. Press release, 18 December 2025, as syndicated by The Globe and Mail, 2025. URL <https://www.theglobeandmail.com/investing/markets/markets-news/ACCESS%20Newswire/36691510/open-navigation-and-3laws-demonstrate-capabilities-of-nav2-and-3laws-supervisor-integration/>. Accessed 2026-09-30.
- A. D. Ames, X. Xu, J. W. Grizzle, and P. Tabuada. Control Barrier Function Based Quadratic Programs for Safety Critical Systems. *IEEE Transactions on Automatic Control*, 62(8):3861–3876, 2017. doi: 10.1109/TAC.2016.2638961. URL <https://doi.org/10.1109/TAC.2016.2638961>.
- A. D. Ames, S. Coogan, M. Egerstedt, G. Notomista, K. Sreenath, and P. Tabuada. Control Barrier Functions: Theory and Applications. In *2019 18th European Control Conference (ECC)*, pages 3420–3431, 2019. doi: 10.23919/ECC.2019.8796030. URL <https://arxiv.org/abs/1903.11199>.
- M. Asselmeier, D. Ahuja, A. Zaro, A. Abuaish, Y. Zhao, and P. A. Vela. Dynamic Gap: Safe Gap-based Navigation in Dynamic Environments. In *2025 IEEE International Conference on Robotics and Automation (ICRA)*, pages 12870–12876, 2025. doi: 10.1109/ICRA55743.2025.11127543. URL <https://arxiv.org/abs/2210.05022>.
- M. Black, G. Fainekos, B. Hoxha, H. Okamoto, and D. Prokhorov. CBFkit: A Control Barrier Function Toolbox for Robotics Applications. In *2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 12428–12434, 2024. doi: 10.1109/IROS58592.2024.10801431. URL <https://arxiv.org/abs/2404.07158>.
- J. Borquez, L. Raus, Y. U. Ciftci, and S. Bansal. DualGuard MPPI: Safe and Performant Optimal Control by Combining Sampling-Based MPC and Hamilton-Jacobi Reachability. *IEEE Robotics and Automation Letters*, 10(7):6944–6951, 2025. doi: 10.1109/LRA.2025.3568686. URL <https://arxiv.org/abs/2502.01924>.

- L. Busellato, F. Cunico, D. Dall’Alba, M. Emporio, A. Giachetti, R. Muradore, and M. Cristani. Uncertainty Aware-Predictive Control Barrier Functions: Safer Human–Robot Interaction through Probabilistic Motion Forecasting. *Robotics and Autonomous Systems*, 197:105291, 2026. doi: 10.1016/j.robot.2025.105291. URL <https://arxiv.org/abs/2508.20812>.
- Y.-H. Chen, S. Ghorl, A. Adil, E. Feron, and C. Belta. Long-Horizon Geometry-Aware Navigation among Polytopes via MILP-MPC and Minkowski-Based CBFs. arXiv preprint arXiv:2604.00162, 2026. URL <https://arxiv.org/abs/2604.00162>.
- N. Cliff. Dominance statistics: Ordinal analyses to answer ordinal questions. *Psychological Bulletin*, 114(3):494–509, 1993. doi: 10.1037/0033-2909.114.3.494. URL <https://doi.org/10.1037/0033-2909.114.3.494>.
- A. Dixit, L. Lindemann, S. X. Wei, M. Cleaveland, G. J. Pappas, and J. W. Burdick. Adaptive Conformal Prediction for Motion Planning among Dynamic Agents. In *Proceedings of the 5th Annual Learning for Dynamics and Control Conference (L4DC)*, volume 211 of *Proceedings of Machine Learning Research*, pages 300–314, 2023. URL <https://proceedings.mlr.press/v211/dixit23a.html>.
- S. Feng, A. Abuaish, and P. A. Vela. Safer Gap: A Gap-based Local Planner for Safe Navigation with Nonholonomic Mobile Robots. arXiv preprint arXiv:2303.08243, 2023. URL <https://arxiv.org/abs/2303.08243>.
- D. Fox, W. Burgard, and S. Thrun. The Dynamic Window Approach to Collision Avoidance. *IEEE Robotics & Automation Magazine*, 4(1):23–33, 1997. doi: 10.1109/100.580977.
- T. Fuke, M. Endo, K. Honda, and G. Ishigami. DRPA-MPPI: Dynamic Repulsive Potential Augmented MPPI for Reactive Navigation in Unstructured Environments. In *2025 IEEE 21st International Conference on Automation Science and Engineering (CASE)*, pages 961–968, 2025a. doi: 10.1109/CASE58245.2025.11163902. URL <https://arxiv.org/abs/2503.20134>.
- T. Fuke, M. Endo, K. Honda, and G. Ishigami. Towards Local Minima-free Robotic Navigation: Model Predictive Path Integral Control via Repulsive Potential Augmentation. In *2025 IEEE/SICE International Symposium on System Integration (SII)*, pages 1112–1117, 2025b. doi: 10.1109/SII59315.2025.10871026. URL <https://arxiv.org/abs/2410.11379>.
- J. Gonzales, K. Mizuta, K. Leung, and L. J. Ratliff. Safe Probabilistic Planning for Human-Robot Interaction using Conformal Risk Control. In *2025 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 18676–18683, 2025. doi: 10.1109/IROS60139.2025.11247339. URL <https://arxiv.org/abs/2603.10392>.
- S. Holm. A Simple Sequentially Rejective Multiple Test Procedure. *Scandinavian Journal of Statistics*, 6(2):65–70, 1979. URL <https://www.jstor.org/stable/4615733>.
- K. Honda, N. Akai, K. Suzuki, M. Aoki, H. Hosogaya, H. Okuda, and T. Suzuki. Stein Variational Guided Model Predictive Path Integral Control: Proposal and Experiments with Fast Maneuvering Vehicles. In *2024 IEEE International Conference on Robotics and Automation (ICRA)*, pages 7020–7026, 2024. doi: 10.1109/ICRA57147.2024.10611021. URL <https://arxiv.org/abs/2309.11040>.
- O. Khatib. Real-Time Obstacle Avoidance for Manipulators and Mobile Robots. *The International Journal of Robotics Research*, 5(1):90–98, 1986. doi: 10.1177/027836498600500106. URL <https://doi.org/10.1177/027836498600500106>.

- T. Kim, R. I. Kee, and D. Panagou. Learning to Refine Input Constrained Control Barrier Functions via Uncertainty-Aware Online Parameter Adaptation. In *2025 IEEE International Conference on Robotics and Automation (ICRA)*, pages 3868–3875, 2025. doi: 10.1109/ICRA55743.2025.11128840. URL <https://arxiv.org/abs/2409.14616>.
- L. Lindemann, M. Cleaveland, G. Shim, and G. J. Pappas. Safe Planning in Dynamic Environments Using Conformal Prediction. *IEEE Robotics and Automation Letters*, 8(8):5116–5123, 2023. doi: 10.1109/LRA.2023.3292071. URL <https://arxiv.org/abs/2210.10254>.
- S. Macenski, F. Martín, R. White, and J. Ginés Clavero. The Marathon 2: A Navigation System. In *2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 2718–2725, 2020. doi: 10.1109/IROS45743.2020.9341207. URL <https://arxiv.org/abs/2003.00368>.
- S. Macenski, S. Singh, F. Martín, and J. Ginés. Regulated Pure Pursuit for Robot Path Tracking. *Autonomous Robots*, 47(6):685–694, 2023. doi: 10.1007/s10514-023-10097-6. URL <https://arxiv.org/abs/2305.20026>.
- H. B. Mann and D. R. Whitney. On a Test of Whether one of Two Random Variables is Stochastically Larger than the Other. *The Annals of Mathematical Statistics*, 18(1):50–60, 1947. doi: 10.1214/aoms/1177730491. URL <https://doi.org/10.1214/aoms/1177730491>.
- Q. McNemar. Note on the Sampling Error of the Difference Between Correlated Proportions or Percentages. *Psychometrika*, 12(2):153–157, 1947. doi: 10.1007/BF02295996. URL <https://doi.org/10.1007/BF02295996>.
- I. S. Mohamed, K. Yin, and L. Liu. Autonomous Navigation of AGVs in Unknown Cluttered Environments: log-MPPI Control Strategy. *IEEE Robotics and Automation Letters*, 7(4):10240–10247, 2022. doi: 10.1109/LRA.2022.3192772. URL <https://arxiv.org/abs/2203.16599>.
- I. S. Mohamed, M. Ali, and L. Liu. GP-Guided MPPI for Efficient Navigation in Complex Unknown Cluttered Environments. In *2023 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 7463–7470, 2023. doi: 10.1109/IROS55552.2023.10341382. URL <https://arxiv.org/abs/2307.04019>.
- A. Nair, F. Jiang, K. Hou, Z. Xu, S. Li, X. Xiao, and P. Stone. DynaBARN: Benchmarking Metric Ground Navigation in Dynamic Environments. In *2022 IEEE International Symposium on Safety, Security, and Rescue Robotics (SSRR)*, pages 347–352. IEEE, 2022. doi: 10.1109/SSRR56537.2022.10018758.
- Nav2 contributors. Enhanced Safety for Nav2 using 3Laws Supervisor. Nav2 documentation, 2026a. URL https://docs.nav2.org/rolling/tutorials/general_tutorials/using_3laws_supervisor/using_3laws_supervisor/. Accessed 2026-09-30.
- Nav2 contributors. Detailed Behavior Tree Walkthrough. Nav2 documentation, 2026b. URL https://docs.nav2.org/rolling/getting_started/nav2_behavior_trees/detailed_behavior_tree_walkthrough/detailed_behavior_tree_walkthrough/. Accessed 2026-09-30.
- Nav2 contributors. nav2_controller, branch jazzy: controller_server.cpp and simple_progress_checker.cpp. Nav2 source code, GitHub repository ros-navigation/navigation2, 2026c. URL https://github.com/ros-navigation/navigation2/tree/jazzy/nav2_controller. Accessed 2026-09-30.

- H. K. Park, T. Kim, and D. Panagou. Beyond Collision Cones: Dynamic Obstacle Avoidance for Nonholonomic Robots via Dynamic Parabolic Control Barrier Functions. arXiv preprint arXiv:2510.01402, 2025. URL <https://arxiv.org/abs/2510.01402>. Accepted to IEEE International Conference on Robotics and Automation (ICRA) 2026.
- H. Parwana and D. Panagou. Rate-Tunable Control Barrier Functions: Methods and Algorithms for Online Adaptation. In *2025 American Control Conference (ACC)*, pages 275–282, 2025. doi: 10.23919/ACC63710.2025.11108110. URL <https://arxiv.org/abs/2303.12966>.
- H. Parwana, M. Black, G. Fainekos, B. Hoxha, H. Okamoto, and D. Prokhorov. Model Predictive Path Integral Methods with Reach-Avoid Tasks and Control Barrier Functions. arXiv preprint arXiv:2407.13693, 2024. URL <https://arxiv.org/abs/2407.13693>.
- H. Parwana, T. Kim, K. Long, B. Hoxha, H. Okamoto, G. Fainekos, and D. Panagou. BR-MPPI: Barrier-Rate Guided MPPI for Enforcing Multiple Inequality Constraints with Learned Signed Distance Fields. arXiv preprint arXiv:2506.07325, 2025. URL <https://arxiv.org/abs/2506.07325>.
- H. Paudel, B. Joshi, D. R. Madai, A. Bartaula, B. Rimal, and S. Neupane. A Closed-Form Dual-Barrier CBF Safety Filter for Holonomic Robots on Incrementally Built Occupancy Grid Maps. arXiv preprint arXiv:2605.05182, 2026. URL <https://arxiv.org/abs/2605.05182>.
- N. Pérez-Higueras, R. Otero, F. Caballero, and L. Merino. HuNavSim: A ROS 2 Human Navigation Simulator for Benchmarking Human-Aware Robot Navigation. *IEEE Robotics and Automation Letters*, 8(11):7130–7137, 2023. doi: 10.1109/LRA.2023.3316072. URL <https://arxiv.org/abs/2305.01303>.
- D. Perille, A. Truong, X. Xiao, and P. Stone. Benchmarking Metric Ground Navigation. In *2020 IEEE International Symposium on Safety, Security, and Rescue Robotics (SSRR)*, pages 116–121, 2020. doi: 10.1109/SSRR50563.2020.9292572. URL <https://arxiv.org/abs/2008.13315>.
- P. Rabiee and J. B. Hoagg. Guaranteed-Safe MPPI Through Composite Control Barrier Functions for Efficient Sampling in Multi-Constrained Robotic Systems. In *2025 IEEE 64th Conference on Decision and Control (CDC)*, pages 5515–5520, 2025. doi: 10.1109/CDC57313.2025.11312129. URL <https://arxiv.org/abs/2410.02154>.
- M. F. Reis, A. P. Aguiar, and P. Tabuada. Control Barrier Function-Based Quadratic Programs Introduce Undesirable Asymptotically Stable Equilibria. *IEEE Control Systems Letters*, 5(2):731–736, 2021. doi: 10.1109/LCSYS.2020.3004797. URL <https://doi.org/10.1109/LCSYS.2020.3004797>.
- C. Rösmann, W. Feiten, T. Wösch, F. Hoffmann, and T. Bertram. Trajectory Modification Considering Dynamic Constraints of Autonomous Robots. In *Proceedings of ROBOTIK 2012; 7th German Conference on Robotics*, pages 74–79. VDE Verlag, 2012. URL <https://ieeexplore.ieee.org/document/6309484>.
- J. Sacks and B. Boots. Learning Sampling Distributions for Model Predictive Control. In *Proceedings of the 6th Conference on Robot Learning (CoRL)*, volume 205 of *Proceedings of Machine Learning Research*, pages 1733–1742, 2022. URL <https://proceedings.mlr.press/v205/sacks23a.html>.
- R. Soloperto, A. Mesbah, and F. Allgöwer. Safe Exploration and Escape Local Minima With Model Predictive Control Under Partially Unknown Constraints. *IEEE Transactions on Automatic Control*, 68(12):7530–7545, 2023. doi: 10.1109/TAC.2023.3283731. URL <https://arxiv.org/abs/2205.03614>.

- B. Stellato, G. Banjac, P. Goulart, A. Bemporad, and S. Boyd. OSQP: An Operator Splitting Solver for Quadratic Programs. *Mathematical Programming Computation*, 12(4):637–672, 2020. doi: 10.1007/s12532-020-00179-2. URL <https://arxiv.org/abs/1711.08013>.
- E. Trevisan and J. Alonso-Mora. Biased-MPPI: Informing Sampling-Based Model Predictive Control by Fusing Ancillary Controllers. *IEEE Robotics and Automation Letters*, 9(6):5871–5878, 2024. doi: 10.1109/LRA.2024.3397083. URL <https://arxiv.org/abs/2401.09241>.
- R. Walia and K. Leahy. Conflict-Aware Switching for CBF-CLF-Based Multi-Goal Navigation. arXiv preprint arXiv:2606.21577, 2026. URL <https://arxiv.org/abs/2606.21577>. CDC 2026 submission copy.
- Z. Wang, O. So, J. Gibson, B. Vlahov, M. S. Gandhi, G.-H. Liu, and E. A. Theodorou. Variational Inference MPC using Tsallis Divergence. In *Robotics: Science and Systems XVII*, 2021. doi: 10.15607/RSS.2021.XVII.073. URL <https://arxiv.org/abs/2104.00241>.
- G. Williams, P. Drews, B. Goldfain, J. M. Rehg, and E. A. Theodorou. Information-Theoretic Model Predictive Control: Theory and Applications to Autonomous Driving. *IEEE Transactions on Robotics*, 34(6):1603–1622, 2018. doi: 10.1109/TRO.2018.2865891. URL <https://arxiv.org/abs/1707.02342>.
- E. B. Wilson. Probable Inference, the Law of Succession, and Statistical Inference. *Journal of the American Statistical Association*, 22(158):209–212, 1927. doi: 10.1080/01621459.1927.10502953. URL <https://doi.org/10.1080/01621459.1927.10502953>.
- W. Xiao, C. Belta, and C. G. Cassandras. Adaptive Control Barrier Functions for Safety-Critical Systems. arXiv preprint arXiv:2002.04577, 2020. URL <https://arxiv.org/abs/2002.04577>.
- X. Xiao, Z. Xu, A. Datar, G. Warnell, P. Stone, J. J. Damanik, J. Jung, C. A. Deresa, T. D. Huy, C. Jinyu, C. Yichen, J. A. Cahyono, J. Wu, L. Mo, M. Lv, B. Lan, Q. Meng, W. Tao, and L. Cheng. Autonomous Ground Navigation in Highly Constrained Spaces: Lessons Learned From the Third BARN Challenge at ICRA 2024. *IEEE Robotics & Automation Magazine*, 31(3):197–204, 2024. doi: 10.1109/MRA.2024.3427891. URL <https://arxiv.org/abs/2407.01862>.
- R. Xu, S. Feng, and P. A. Vela. Potential Gap: Using Reactive Policies to Guarantee Safe Navigation. arXiv preprint arXiv:2103.11491, 2021. URL <https://arxiv.org/abs/2103.11491>.
- Y. Xue and N. Figueroa. No Minima, No Collisions: Combining Modulation and Control Barrier Function Strategies for Feasible Dynamic Collision Avoidance. arXiv preprint arXiv:2502.14238, 2025. URL <https://arxiv.org/abs/2502.14238>.
- Y. Xue, Z. Zhang, K. Åkesson, and N. Figueroa. Proactive Local-Minima-Free Robot Navigation: Blending Motion Prediction With Safe Control. *IEEE Robotics and Automation Letters*, 11(4):5096–5103, 2026. doi: 10.1109/LRA.2026.3668447. URL <https://arxiv.org/abs/2601.10233>.
- J. Yin, C. Dawson, C. Fan, and P. Tsiotras. Shield Model Predictive Path Integral: A Computationally Efficient Robust MPC Method Using Control Barrier Functions. *IEEE Robotics and Automation Letters*, 8(11):7106–7113, 2023. doi: 10.1109/LRA.2023.3315211. URL <https://arxiv.org/abs/2302.11719>.
- J. Zeng, B. Zhang, Z. Li, and K. Sreenath. Safety-Critical Control using Optimal-decay Control Barrier Function with Guaranteed Point-wise Feasibility. In *2021 American Control Conference (ACC)*, pages 3856–3863, 2021. doi: 10.23919/ACC50511.2021.9482626. URL <https://arxiv.org/abs/2103.12375>.

H. Zhou, Y. Zhang, and W. Luo. Safety-Critical Control with Uncertainty Quantification using Adaptive Conformal Prediction. In *2024 American Control Conference (ACC)*, pages 574–580, 2024. doi: 10.23919/ACC60939.2024.10644391. URL <https://arxiv.org/abs/2407.03569>.

A. Reproduction

All paths are relative to the code release (<https://github.com/kjungmo/se-mppi-nav2>, Apache-2.0).

Software. ROS 2 Jazzy, Nav2 and Gazebo are installed through RoboStack (conda channels `robostack-jazzy` and `conda-forge`) by `scripts/setup_ros2_env.sh`, which also installs `osqp-eigen` and `eigen` for the CBF-QP and a compiler toolchain, so the build does not depend on the host OS version.

Benchmark regeneration. Every benchmark number is generated from the committed per-trial file by committed scripts. The committed data use $N = 50$:

```
export MAMBA_ROOT_PREFIX=$HOME/micromamba
micromamba run -n ros2 python3 -m experiments.benchmark2d.runner \
    --families utrap clutter dynamic narrowdyn -n 50 --workers 6 --max-steps 400
micromamba run -n ros2 python3 -m experiments.benchmark2d.report
```

The figures and tables of this paper are regenerated from those outputs, and the mechanism runs of Table 2 are re-executed, by

```
micromamba run -n ros2 python3 docs/papers/arxiv_ref/scripts/make_figures.py
python3 docs/papers/arxiv_ref/scripts/make_tables.py
python3 scripts/check_paper_numbers.py --arxiv
```

Scenarios and seeds. Seeds 0–49 per family; the generators in `experiments/benchmark2d/scenarios.py` seed numpy generators with a per-family offset plus the trial seed (10^6 U-trap, 2×10^6 clutter, 3×10^6 dynamic, 4×10^6 narrow-dynamic), and the MPPI noise stream uses the same trial seed (`experiments/benchmark2d/runner.py`), so every configuration sees identical worlds and rollout noise. The six toggle sets are in `experiments/benchmark2d/configs.py`; the overlays for the deferred full-scale protocol are in `experiments/configs/ablations.yaml`.

Data. Raw per-trial data: `experiments/results_2d/trials.csv` (1,200 rows). Aggregates and statistics: `summary.csv`, `stats.json`, `tables.md` in the same directory. Live pilot telemetry and logs: `experiments/results_pilot/`. The claim-to-artifact map for this paper is `docs/papers/arxiv_ref/NUMBERS.md`.

B. Parameters

C. Differences Between the 2D Mirror and the Nav2 Controller

The 2D mirror (`experiments/prototype/se_mppi_proto.py`, driven by `experiments/benchmark2d/rollout.py`) reuses the controller’s formulas but differs in the following documented ways.

- *Progress signal.* Without a global path, entrapment is declared when the best distance-to-goal has not improved by 10^{-3} m for $W = 15$ cycles, instead of the stall of $k_t^{\max}$ in (3).
- *Gap use.* The selected gap bearing places a temporary subgoal 2.5 m along the opening, replacing the goal in the MPPI cost while entrapped, in lieu of Nav2’s global routing and the critic’s C_{gap} term; the gap choice carries a hysteresis term toward the previous bearing so that symmetric openings do not flip every cycle.

Table 5: Parameters. Released-controller defaults come from the package’s parameter file and parameter getters; 2D-mirror values from the prototype primitives and the benchmark rollout (Sections A and C).

Symbol	Meaning	Released controller	2D mirror
$T, \Delta t$	MPPI horizon steps, step (s)	56, 0.05	25, 0.1
K	rollouts per cycle	1000	600
λ	MPPI temperature	0.3	0.3
$\Sigma^{1/2}$	sampling std. of (ν, ω)	(0.2, 0.4)	(0.25, 0.6)
$\mathcal{U}$	$\nu \in [\nu_{\min}, \nu_{\max}]$, $ \omega \leq \omega_{\max}$	$[-0.35, 0.5]$, 1.9	$[-0.35, 0.5]$, 1.9
W	entrapment stall window (cycles)	30	15
ϵ_{goal}	entrapment suppression radius (m)	0.5	–
d_0, η	APF influence distance (m), gain	0.6, 1.0	0.8, 0.3
$N_r, r_{\max}, r_{\min}$	gap rays, ray range (m), min. clearance (m)	36, 2.0, 0.6	72, 3.0, 2.4
w_g	gap-attraction weight	4.0	subgoal (Section C)
L	look-ahead distance (m)	0.2	0.2
r, m	robot radius (m), safety margin (m)	0.225 (logged), 0.05	0.22, 0.05
ρ	QP slack weight	10^3	10^3
ϵ_δ	slack threshold for braking	10^{-3}	10^{-3}
N_{obs}	obstacles kept in the QP	20	all dynamic
$\alpha_{\text{base}}, \alpha_{\text{escape}}$	CBF class- $\mathcal{K}$ gains	2.0, 6.0	2.0, 6.0
τ	TTC override threshold (s)	1.5	1.5
$v_{\text{dyn}}, R_{\max}$	CBF admission: min. speed (m/s), max. radius (m)	0.1, 1.0	$\ \mathbf{v}_o\ > 0$

- *Geometry.* Obstacles, including walls built from chains of discs, are circles; APF distances and ray lengths are computed analytically against them rather than on a costmap distance field.
- *Obstacle knowledge.* The CBF and TTC receive the true positions and velocities of the moving obstacles, so the tracker of Section 3.4 is not exercised.
- *Costs.* The MPPI cost is a goal-distance, path-progress and obstacle-proximity cost rather than Nav2’s critic stack; parameters differ as listed in Table 5.
- *Mechanism scenes.* Table 2 comes from `run_validation.py`, whose CBF sees all obstacles, including the static wall discs; the benchmark mirror restricts the CBF to movers, as the Nav2 controller does.

D. Extended Results

Table 6 lists every configuration and family of the benchmark, and Table 7 the slack usage behind the discussion of Sections 3.6 and 4.3. Configurations E and F coincide exactly on U-trap and clutter because the CBF sees no obstacle in those mover-free families, so the gain schedule is irrelevant there.

E. Notation

Table 6: Full per-family results of the Python 2D-mirror benchmark ($N = 50$ per cell). Rates in %; time-to-goal and path length are means over successful trials only ($\pm 95\%$ CI half-width); minimum clearance is over all trials. Source: `experiments/results_2d/summary.csv`.

Family	Config	Succ.	Coll.	Timeout	Time-to-goal (s)	Path (m)	Min. clear. (m)
U-trap	A	0	0	100	–	–	0.33 $\pm$ 0.00
	C	90	0	10	31.34 $\pm$ 1.36	8.12 $\pm$ 0.33	0.32 $\pm$ 0.00
	D	0	0	100	–	–	0.33 $\pm$ 0.00
	E	88	0	12	31.20 $\pm$ 1.36	8.09 $\pm$ 0.32	0.32 $\pm$ 0.00
	F	88	0	12	31.20 $\pm$ 1.36	8.09 $\pm$ 0.32	0.32 $\pm$ 0.00
	F [–]	0	0	100	–	–	0.33 $\pm$ 0.00
Clutter	A	52	0	48	15.23 $\pm$ 0.49	5.81 $\pm$ 0.17	0.36 $\pm$ 0.02
	C	62	0	38	17.67 $\pm$ 2.11	6.14 $\pm$ 0.32	0.35 $\pm$ 0.02
	D	52	0	48	15.23 $\pm$ 0.49	5.81 $\pm$ 0.17	0.36 $\pm$ 0.02
	E	62	0	38	17.67 $\pm$ 2.11	6.14 $\pm$ 0.32	0.35 $\pm$ 0.02
	F	62	0	38	17.67 $\pm$ 2.11	6.14 $\pm$ 0.32	0.35 $\pm$ 0.02
	F [–]	52	0	48	15.23 $\pm$ 0.49	5.81 $\pm$ 0.17	0.36 $\pm$ 0.02
Dynamic	A	80	20	0	22.48 $\pm$ 1.44	7.99 $\pm$ 0.55	0.12 $\pm$ 0.03
	C	76	20	4	21.87 $\pm$ 1.36	7.75 $\pm$ 0.53	0.13 $\pm$ 0.03
	D	82	16	2	22.74 $\pm$ 1.52	8.02 $\pm$ 0.65	0.13 $\pm$ 0.03
	E	78	18	4	21.77 $\pm$ 1.25	7.56 $\pm$ 0.47	0.13 $\pm$ 0.03
	F	78	18	4	21.78 $\pm$ 1.25	7.57 $\pm$ 0.47	0.13 $\pm$ 0.03
	F [–]	84	16	0	23.01 $\pm$ 1.66	8.14 $\pm$ 0.68	0.13 $\pm$ 0.03
Narrow-dyn.	A	0	18	82	–	–	0.22 $\pm$ 0.04
	C	78	18	4	30.52 $\pm$ 1.30	8.33 $\pm$ 0.44	0.21 $\pm$ 0.04
	D	0	32	68	–	–	0.19 $\pm$ 0.04
	E	64	32	4	30.55 $\pm$ 1.50	8.24 $\pm$ 0.55	0.18 $\pm$ 0.04
	F	62	32	6	30.36 $\pm$ 1.51	8.11 $\pm$ 0.50	0.18 $\pm$ 0.04
	F [–]	0	32	68	–	–	0.19 $\pm$ 0.04

Table 7: CBF slack usage in the Python 2D mirror. Trials (of 50) in which the QP relaxed a barrier row at least once, and the largest per-trial `slack_max`. Configurations without a CBF (A, C) are zero by construction; on the mover-free families the CBF sees no obstacle. Source: `experiments/results_2d/stats.json` (`slack_usage`).

Family	A	C	D	E	F	F [–]
U-trap	0	0	0	0	0	0
Clutter	0	0	0	0	0	0
Dynamic	0	0	47 (0.015)	48 (0.015)	48 (0.015)	47 (0.015)
Narrow-dyn.	0	0	26 (0.469)	34 (0.469)	34 (0.469)	26 (0.469)

Table 8: Notation.

Symbol	Meaning
$\mathbf{x}_t = (x_t, y_t, \theta_t), \mathbf{p}_t$	robot pose, robot position
$\mathbf{u}_t = (v_t, \omega_t), \mathcal{U}$	control (linear, angular velocity), admissible box
$\sigma = (\sigma_1, \dots, \sigma_M)$	global path from the Nav2 planner
$k_t, k_t^{\max}$	nearest and furthest-reached path index
s_t, W, e_t	stall counter, stall window, entrapment flag
$O_t; \mathbf{p}_o, R_o, \mathbf{v}_o$	tracked dynamic obstacles; position, radius, velocity
$\mathbf{P}_t, L, G(\theta)$	look-ahead point, its offset, its Jacobian
h_o, R_o^{eff}	barrier for obstacle o , effective radius $r + R_o + m$
$\alpha_t; \alpha_{\text{base}}, \alpha_{\text{escape}}$	class- $\mathcal{K}$ gain; its two scheduled values
$\delta, \rho, \epsilon_\delta$	QP slack, slack weight, braking threshold
TTC_t, τ	minimum time-to-collision, override threshold
S^k, w^k, λ	rollout cost, softmax weight, temperature
$C_{\text{apf}}, C_{\text{gap}}$	escape costs (repulsion, gap attraction)
$\psi^\star$	bearing of the selected free-space gap